

第12课 vLLM V1 系统启动过程

本文回答一个系统级问题：

从执行 `vllm serve` 开始，到 HTTP Server 真正可以接收请求，API Server、EngineCore、Executor、Worker、KV Cache 和 Scheduler 按什么顺序完成初始化？

一、API Server 准备启动配置

阶段 0：CLI 选择服务入口

`cli.main()` 注册子命令时，把各子命令的 `cmd` 函数保存为对应 parser 的默认值；解析到 `serve` 后，`args.dispatch_function` 才会指向 `ServeSubcommand.cmd`。

- `vllm serve <model> [options]`
 - `vllm.entrypoints.cli.main.main()`
 - i. `FlexibleArgumentParser(...)` 创建顶层 parser。
 - ii. `cmd_module.cmd_init()` 返回各子命令对象；`serve.cmd_init()` 返回 `[ServeSubcommand()]`。
 - iii. `ServeSubcommand.subparser_init()` 创建 serve parser，并调用 `make_arg_parser()`。
 - iv. `cmd.subparser_init(...).set_defaults(dispatch_function=cmd.cmd)` 把 `ServeSubcommand.cmd` 保存到 serve parser 的 defaults 中。
 - v. `parser.parse_args()` 解析到 `serve`，因此产生的 Namespace 满足 `args.dispatch_function is ServeSubcommand.cmd`。
 - vi. `ServeSubcommand.validate(args)` 调用 `validate_parsed_serve_args()`。
 - vii. `args.dispatch_function(args)`
 - 此时等价于调用 `ServeSubcommand.cmd(args)`。

`ServeSubcommand.cmd()` 不是简单地直接调用 `run_server()`。gRPC 会先行返回；其余路径再确定 API Server 数量和部署模式，并选择真正的入口：

- `ServeSubcommand.cmd(args)`
 - `grpc` → `serve_grpc()`

- 先计算 `api_server_count` 与 LB 模式
- `is_multi_port` → `run_dp_supervisor()`
- `api_server_count < 1` → `run_headless()`
- `api_server_count > 1` 或启用 Rust frontend → `run_multi_api_server()`
- 其他情况 → `uvloop.run(run_server(args))`，这是本文主线

External LB 和 Hybrid LB 会参与 `api_server_count` 的计算，但不等价于直接进入 `run_dp_supervisor()`；只有 multi-port 模式直接选择 supervisor。

阶段 1：创建监听 socket 和 VllmConfig

单 API Server 分支执行 `uvloop.run(run_server(args))`。`run_server()` 先用 `setup_server()` 绑定 socket，再 `await run_server_worker(...)`。

- `ServeSubcommand.cmd()` 的单 API Server 分支
 - `uvloop.run(run_server(args))`
 - `run_server(args)`
 1. `listen_address, sock = setup_server(...)` → listening socket。
 2. `await run_server_worker(listen_address, sock, args)`
 - `run_server_worker()`
 - `async with build_async_engine_client(args)`
 1. `AsyncEngineArgs.from_cli_args(args)`。
 2. `async with build_async_engine_client_from_engine_args(...)`
 - `vllm_config = engine_args.create_engine_config(...)` → `VllmConfig`；方法定义见 `AsyncEngineArgs.create_engine_config()`。

这里要区分两个配置对象：

- `AsyncEngineArgs` 基本是 CLI 参数的对象化表示。
- `VllmConfig` 是经过校验、默认值补齐和派生计算后的系统配置。

此时 listening socket 已经绑定，因此端口可能已经可见；但代码尚未进入 `serve_http()`，应用也没有开始处理用户请求。端口已经绑定不等于服务已经 Ready。

阶段 2：进入 AsyncLLM 和 EngineCoreClient 构造

`run_server_worker()` 进入外层 context manager; 外层再进入 `build_async_engine_client_from_engine_args()`。真正调用 `AsyncLLM.from_vllm_config()` 的是后者, 而不是 `run_server_worker()` 直接调用。

- `run_server_worker()`
 - `async with build_async_engine_client(args)`
 - `engine_args = AsyncEngineArgs.from_cli_args(args)`
 - `async with build_async_engine_client_from_engine_args(engine_args)`
 - `vllm_config = engine_args.create_engine_config(...)`
 - `async_llm = AsyncLLM.from_vllm_config(...)` ← 实际调用点
 - `AsyncLLM.from_vllm_config()`
 - `Executor.get_class(vllm_config)` → 选择 UniProc、Multiproc、Ray 或 external launcher。
 - `AsyncLLM.__init__(...)`
 - `renderer_from_config(vllm_config)`。
 - `InputProcessor(vllm_config, renderer)`。
 - `OutputProcessor(tokenizer, ...)`。
 - `self.engine_core = EngineCoreClient.make_async_mp_client(...)`
 - `EngineCoreClient.make_async_mp_client()`
 - DP external LB → `DPAAsyncMPCClient`
 - DP internal LB → `DPLBAsyncMPCClient`
 - DP=1 → `AsyncMPCClient` , 这是本文主线

`EngineCoreClient.make_async_mp_client()` 是同步构造过程。调用进入这里以后, `AsyncLLM.__init__()` 不会立即返回, 而是一直等到 `EngineCore` 完成初始化并返回最终 Ready 信息。

二、EngineCore 和 Worker 进程启动

阶段 3: 启动 EngineCore 并进入 HELLO/READY 握手

`AsyncLLM.__init__()` 建好 Frontend processor 后构造 `EngineCoreClient`。DP=1 主线进入 `AsyncMPCClient` / `MPCClient`, 并在 `MPCClient.__init__()` 内启动 `EngineCore`。

API 进程创建正式通信通道

- `AsyncLLM.__init__()`
 - `EngineCoreClient.make_async_mp_client(...)` 调用点
 - `make_async_mp_client()`
 - DP=1: `return AsyncMPClient(*client_args)`
 - `AsyncMPClient.__init__()`
 - `super().__init__(asyncio_mode=True, ...)` → `MPCClient.__init__()`
 - 创建 ZMQ Context
 - `get_engine_zmq_addresses(vllm_config)`
 - bind input ROUTER socket
 - 创建 output PULL socket
 - `with launch_core_engines(...)`

`MPCClient.__init__()` 建立这些正式通信通道。这里的命名是从 Frontend 的用途出发：

- input ROUTER: Frontend 向 EngineCore 发送请求，也接收 EngineCore 建连时发来的第一条 Ready payload。
- output PULL: Frontend 接收 EngineCore 的请求输出。
- handshake socket: 只用于 EngineCore 进程启动期的 HELLO/READY 协议。

创建 EngineCore 子进程

`launch_core_engines()` 在 context 内创建 `CoreEngineProcManager`，后者把 `EngineCoreProc.run_engine_core()` 设置为子进程入口：

- `launch_core_engines(...)`
 - `local_engine_manager = CoreEngineProcManager(...)`
 - `CoreEngineProcManager.__init__()`
 - `Process(target=EngineCoreProc.run_engine_core)`
 - `process.start()`
 - 跨进程进入 `EngineCoreProc.run_engine_core()`

HELLO/READY 握手

两端进入握手函数的直接路径是：

- EngineCore Process: `EngineCoreProc.__init__()`
 - `with self._perform_handshakes(...)`
 - DP=1 主线进入 `self._perform_handshake(...)`
 - `startup_handshake(...)`
- API Server Process: `MPCClient.__init__()` 的 `with launch_core_engines(...)`
 - `with` 主体结束, `launch_core_engines()` 从 `yield` 后继续
 - `wait_for_engine_startup(...)`

EngineCore Process: <code>_perform_handshake()</code>	API Server Process: <code>wait_for_engine_startup()</code>
发送 <code>{status: HELLO}</code>	接收 HELLO
等待初始化 metadata	发送 <code>EngineHandshakeMetadata</code> , 其中包含 ZMQ addresses 和必要的 DP config
接收 metadata	等待 EngineCore 完成主体初始化
在 handshake context 内构造 Executor、Worker、KV Cache 和 Scheduler	仍在 <code>wait_for_engine_startup()</code> 中阻塞
context 退出并发送 <code>{status: READY}</code>	接收 READY, <code>wait_for_engine_startup()</code> 返回

`HELLO` 的作用是登记 EngineCore 身份并取得后续正式通信所需的地址。`wait_for_engine_startup()` 接收 HELLO 并回传 `EngineHandshakeMetadata` ; EngineCore 的 Executor、Worker、KV Cache 和 Scheduler 初始化都发生在 `_perform_handshake()` 的 context 内部, context 退出时才发送 handshake READY。

因此阶段 3 只是发起握手并进入等待; 这层 `with launch_core_engines(...)` 要到阶段 10 收到 handshake READY 后才真正返回。阶段 4~9 都嵌在这次尚未完成的 EngineCore 构造中。

阶段 4：进入 Executor 构造并确定 Worker 拓扑

EngineCore 子进程从 `run_engine_core()` 进入 `EngineCoreProc(...)` ; 其 handshake context 内部通过 `EngineCore.__init__()` 构造 Executor。

- `EngineCoreProc.run_engine_core()`

- `engine_core = EngineCoreProc(...)`
 - `EngineCoreProc.__init__()`
 - with `_perform_handshakes(...)` as addresses
 - `_init_data_parallel(vllm_config)`
 - `super().__init__(...)` → `EngineCore.__init__()`
 - `self.model_executor = executor_class(vllm_config)`
 - `Executor.__init__()`
 - `self._init_executor()` 动态进入具体 Executor 实现。

Executor 决定 Worker 的执行拓扑。

UniProcExecutor

- `UniProcExecutor._init_executor()`
 - `WorkerWrapperBase(rpc_rank=0)`
 - `driver_worker.init_worker(...)`
 - `driver_worker.init_device()`
 - `driver_worker.load_model()`

`UniProcExecutor._init_executor()` 直接构造 `WorkerWrapperBase` 并调用 Worker。当 `world_size=1` 且没有显式选择其他 backend 时，v0.25.1 会把 `distributed_executor_backend` 设置为 `uni`。此时不存在独立 Worker OS 进程。

MultiprocExecutor

- `MultiprocExecutor._init_executor()`
 - a. 为每个 `local_rank` 调用 `WorkerProc.make_worker_process(...)`
 - `multiprocessing.Process(target=WorkerProc.worker_main)`，进程入口是 `WorkerProc.worker_main()`。
 - 分别进入 Worker Process rank 0、rank 1.....
 - b. `WorkerProc.wait_for_ready(all_workers)`。
 - c. 等待 broadcast / response MessageQueue ready。

`MultiprocExecutor._init_executor()` 会先创建所有 Worker，再统一等待 READY，避免某个 Worker 在 `init_device()` 的设备同步阶段造成创建顺序死锁。

阶段 5: Worker 初始化设备并加载模型

`MultiprocExecutor._init_executor()` 为每个 rank 启动 `WorkerProc`；Worker 在自己的进程中依次完成 `init_device()`、`load_model()` 和消息队列初始化，然后通过 ready pipe 返回队列句柄。

Multiproc 路径中，每个 Worker 子进程执行：

- `MultiprocExecutor._init_executor()`
 - `WorkerProc.make_worker_process(...)` 调用点
 - `WorkerProc.make_worker_process()`
 - `Process(target=WorkerProc.worker_main)`
 - 跨进程进入 `WorkerProc.worker_main()`
 - a. `worker = WorkerProc(...)`
 - `WorkerProc.__init__()`
 - `WorkerWrapperBase(rpc_rank=local_rank, global_rank=rank)`
 - `wrapper.init_worker(all_kwargs)`
 - `self.worker = worker_class(**kwargs)` → `gpu_worker.Worker`
 - `wrapper.init_device()` → `gpu_worker.Worker.init_device()`
 - 选择 CUDA device
 - 初始化 TP、PP、NCCL 等分布式环境
 - 设置随机种子并清理 allocator cache
 - `MemorySnapshot(device)`
 - 创建 `GPUModelRunner`
 - `wrapper.load_model()` → `gpu_worker.Worker.load_model()`
 - `GPUModelRunner.load_model()` → 构造模型、加载权重并完成加载后处理；内部调用在阶段 5.1~5.3 展开。
 - `_init_message_queues(...)`
 - b. `ready_writer.send({status: READY, ...})`
 - c. `MessageQueue.wait_until_ready()`
 - d. `worker.worker_busy_loop()`

- EngineCore 进程中的 `WorkerProc.wait_for_ready(all_workers)` 收到所有 READY。
- `MultiprocExecutor._init_executor()` 再等待 broadcast / response MessageQueue ready, 之后才返回。

顺序上最重要的是: `gpu_worker.Worker.init_device()` 在 `MemorySnapshot` 之前初始化分布式环境和 NCCL 资源。这样后续计算 KV Cache 可用显存时, NCCL 等固定开销已经反映在初始显存快照中。

`WorkerProc.worker_main()` 在 Worker 构造完成后发送 READY, 其中携带 response queue handles。它并不表示 KV Cache 已完成规划和分配; READY 发出后, Worker 和 Executor 还要完成 MessageQueue 的 `wait_until_ready()` 同步。

阶段 5.1: ModelRunner 选择 Loader 并进入模型加载

下面回到阶段 5 中尚未返回的 `Worker.load_model()`。阶段 5.1~5.3 都发生在这次调用内部, 早于 `_init_message_queues()` 和 WorkerProc READY, 不是 READY 之后新增的操作。

- `gpu_worker.Worker.load_model()`
 - `self.model_runner.load_model(...)` 调用点
 - `GPUModelRunner.load_model()` 实现
 1. `model_loader = get_model_loader(self.load_config)` → 选择 Loader。
 2. `self.model = model_loader.load_model(...)` → 等待构造、权重加载和加载后处理完成, 才赋值给 Runner 的 `self.model`。

V2 ModelRunner 的对应调用点也调用 `get_model_loader()` 和 `model_loader.load_model()`, 下面的普通 Loader / Llama 构造链可以共用。

这里固定以普通 Llama 文本生成、`load_format="auto"` 的路径为例: `get_model_loader()` 根据格式选择 Loader; `auto` 对应 `DefaultModelLoader`。特殊加载器可以覆盖加载过程, 不在此处逐项展开。

- `model_loader.load_model(...)` ← 接上本阶段的调用点。
 - `BaseModelLoader.load_model()` ← `DefaultModelLoader` 继承的实现。
 - i. `initialize_model(...)` → 构造模型对象和内部各层, 阶段 5.2 展开。
 - ii. `self.load_weights(model, model_config)` → 将 checkpoint 权重加载到已构造的模型, 阶段 5.3 展开。
 - iii. 若启用 online quantization, 执行 `finalize_layerwise_processing(...)`。

- iv. `process_weights_after_loading(...)` → 完成量化方法、Attention 等模块需要的加载后处理。
- v. `return model.eval()` → 返回推理模式的模型对象，不执行 forward。

Loader 在指定的默认 dtype 和目标设备上下文中构造模型。构造模型不只是创建 Python 空壳，还会建立参数及相关存储；加载 checkpoint 则为这些参数填入训练得到的权重。

阶段 5.2：从模型对象逐层构造到 Attention

上一阶段停在 `BaseModelLoader.load_model()` 的第一步：`model = initialize_model(...)`。先选出模型类，再沿模型内部的包含关系构造对象。

创建外层模型和 LlamaModel

- `initialize_model()`
 - `get_model_architecture(model_config)` → 本例选出 `LlamaForCausalLM`。
 - `model_class(vllm_config=..., prefix=...)` 调用点
 - `LlamaForCausalLM.__init__()`
 - `self._init_model(...)` 调用点
 - `_init_model()` 实现：`return LlamaModel(...)`。

这里的 `LlamaForCausalLM.model` 和前面 `GPUModelRunner.model` 不是同一个成员：Runner 持有整个 `LlamaForCausalLM`，后者的 `model` 持有内部的 `LlamaModel`。外层模型还会按 PP rank 和配置构造或绑定 `lm_head`；下面只沿 Decoder 层这条支线继续。

LlamaModel 创建当前 Worker 的 Decoder 层

`_init_model()` 执行 `LlamaModel(...)`，进入其构造函数。这里不再重复 Loader 和外层模型，而是从这个对象构造点继续。

- `LlamaModel.__init__()`
 - `make_layers(..., layer_fn, ...)` 调用点
 - `make_layers()` 实现
 - `layer_fn(prefix=f"{prefix}.{idx}")` 调用点
 - 执行传入的 `lambda prefix: layer_type(...)`；默认 `layer_type=LlamaDecoderLayer`，因此进入 `LlamaDecoderLayer.__init__()`。

`lambda` 是“创建一层的方法”，传给 `make_layers()` 时还没有创建 Decoder 层；`make_layers()` 按当前 PP rank 的层范围调用它，其他层的位置放 `PPMissingLayer`。例如当前 Worker 负责

第 0 层时，这次调用会使用 `prefix="model.layers.0"`。

Decoder 层创建 LlamaAttention，后者创建 Attention

接上 `layer_fn(...)` 创建的那个 Decoder 层。构造参数中的默认类决定了下面两次动态调用的实际目标。

- `LlamaDecoderLayer.__init__()`
 - `self.self_attn = attn_layer_type(...)` 调用点，默认 `attn_layer_type=LlamaAttention`。
 - `LlamaAttention.__init__()`
 - 创建 `self.qkv_proj`、`self.o_proj`，调用 `_init_rotary_emb(...)` 创建 RoPE。
 - `self.attn = attn_cls(...)` 调用点，本例普通 decoder 路径中 `attn_cls=Attention`。
 - 进入 `Attention.__init__()`，建立后端实现、层注册信息和 KV Cache 占位。

`LlamaAttention` 组织投影、RoPE 和核心 Attention；其 `attn` 成员才是负责 KV 读写与核心计算的 `Attention` 对象。此时只是构造这些模块，还没有用请求的 Q/K/V 执行计算。

在 `Attention.__init__()` 内，三个与后续启动阶段直接相关的动作是：

1. `get_attn_backend(...)` 选择后端类，再通过 `get_impl_cls()` 和 `impl_cls(...)` 创建 `self.impl`；这里没有执行 Attention kernel。
2. `static_forward_context[prefix] = self` 按层名登记对象。本例第 0 层的名字是 `model.layers.0.self_attn.attn`。
3. `self.kv_cache = torch.tensor([])` 只建立占位。真实 KV 存储要等阶段 6~8 完成 Spec 收集、容量规划和 Tensor 分配后，再绑定到这些已存在的 Attention 对象。

各层构造结束后，调用逐层返回到 `initialize_model()`，Loader 才继续下面的权重加载。后续 KV Cache 初始化找到的是这里创建的 Attention 对象，不是重新创建一套模型层。

阶段 5.3：加载权重并返回模型

阶段 5.2 返回的是已构造的模型。调用栈回到 `BaseModelLoader.load_model()`，从 `self.load_weights(model, model_config)` 继续。

- `self.load_weights(model, model_config)` ← Loader 内部调用点。
 - `DefaultModelLoader.load_weights()`

- `model.load_weights(self.get_all_weights(model_config, model))` 调用点
 - `LlamaForCausalLM.load_weights()`
 - 创建 `AutoWeightsLoader(self, ...)`，调用 `loader.load_weights(weights)`，将权重分发给模型内部模块。

这里传入的是权重迭代器：文件读取随迭代推进，不是先在此处生成一个完整权重列表再统一加载。参数名映射、融合权重和 TP 切片属于下一层加载实现，本节只保留它们进入模型的入口。

`DefaultModelLoader.load_weights()` 返回后，执行流回到 `BaseModelLoader.load_model()`：按需完成 online quantization 的收尾，再调用 `process_weights_after_loading()`，让量化方法完成必要的布局处理，并完成 Attention 的加载后处理，最后返回 `model.eval()`。

Runner 的 `self.model` 此时才取得 Loader 返回的模型；Runner 完成其余配置相关的准备后，返回 `Worker.load_model()`。随后阶段 5 的 `WorkerProc.__init__()` 继续创建消息队列，Worker 主函数发送 READY，并进入队列就绪同步。

因此，这一段的终点是：模型对象、各层后端实现和权重已经准备好，但请求尚未执行，正式 KV Cache 存储尚未分配。所有 Worker 和队列就绪后，EngineCore 才进入阶段 6 的资源规划。

公共机制：后续 Executor → Worker RPC 共用的分发链

MessageQueue 就绪后，启动期的 KV Cache 查询、显存测量、物理初始化，以及后续运行期的 Worker 方法，都复用同一条 RPC 基础设施：

```

1  Executor public method
2  → MultiprocExecutor.collective_rpc(method, args, output_rank)
3  → rpc_broadcast_mq.enqueue(...)
4  → every WorkerProc.worker_busy_loop().dequeue()
5  → getattr(WorkerWrapperBase, method)(*args, **kwargs)
6  → wrapper method or delegated real Worker method
7  → selected response queue / aggregated responses

```

实现位置：

- `MultiprocExecutor.collective_rpc()`
负责广播 method/args，并根据 `output_rank` 或 aggregator 收集响应。
- `WorkerProc.worker_busy_loop()`
负责 dequeue、动态查找方法和执行。
- `WorkerWrapperBase.__getattr__()`

负责将 wrapper 没有显式实现的方法交给真实 Worker；wrapper 自己实现的方法则先执行 wrapper 逻辑。

后文遇到 `collective_rpc("method")` 时，只说明 method、payload、Worker 终点与返回值，不再重复这条 MessageQueue 分发链。

三、KV Cache 规划、分配与 Scheduler 构造

阶段 6：收集 KVCacheSpec 并测量可用显存

所有 Worker 和 MessageQueue 就绪后，`MultiprocExecutor._init_executor()` 返回，继而使 `executor_class(vllm_config)` 返回。调用栈回到 `EngineCore.__init__()`，下一条资源初始化语句就是 `_initialize_kv_caches()`。

- `EngineCore.__init__()`
 - `kv_cache_config = self._initialize_kv_caches(vllm_config)` 调用点
 - `EngineCore._initialize_kv_caches()`
 1. `register_all_kvcache_specs(vllm_config)`。
 2. `model_executor.get_kv_cache_specs()` 调用点
 - `Executor.get_kv_cache_specs()`
 - `MultiprocExecutor.collective_rpc("get_kv_cache_spec")`
，向每个 WorkerProc 分发方法名。
 - 跨 RPC 进入每个 Worker: `gpu_worker.Worker.get_kv_cache_spec()`
 - `GPUModelRunner.get_kv_cache_spec()` → `dict[layer_name, KVCacheSpec]`。
 3. `model_executor.determine_available_memory()` 调用点
 - `Executor.determine_available_memory()`
 - `MultiprocExecutor.collective_rpc("determine_available_memory")`
 - 跨 RPC 进入每个 Worker: `gpu_worker.Worker.determine_available_memory()`
 - `memory_profiling(...)`
 - `model_runner.profile_run()`

- 可选执行 `profile_cudagraph_memory()`
- 返回 available memory, 单位为 bytes。

`get_kv_cache_specs()` 返回 `list[dict[str, KVCacheSpec]]` :

- 外层列表按 Worker 排列;
- 每个字典按 layer name 索引该 Worker 所持有层的 `KVCacheSpec` 。

`determine_available_memory()` 同样返回每个 Worker 的可用字节数列表, 两者长度必须一致。

两个分支需要单独记住:

- Attention-free 模型没有 KV Cache, EngineCore 为每个 Worker 使用 0 作为 KV 可用内存, 不执行 KV 显存 profiling RPC。
- 显式设置 `kv_cache_memory_bytes` 时, `determine_available_memory()` 仍执行 `profile_run()`, 以完成最大 batch 相关的必要编译, 但跳过自动显存预算计算。

具体 Spec 的生成分别位于 V1 `GPUModelRunner.get_kv_cache_spec()` 和 V2 `GPUModelRunner.get_kv_cache_spec()` 。

阶段 7: 生成每个 Worker 的 KVCacheConfig

两个 Worker RPC 返回后, `_initialize_kv_caches()` 将 `kv_cache_specs` 和 `available_memory` 一起交给 `get_kv_cache_configs()` 。

- `EngineCore._initialize_kv_caches()`
 - a. `kv_cache_configs = get_kv_cache_configs(...)` 调用点
 - `get_kv_cache_configs(...)`
 - 输入 `vllm_config`
 - 输入每个 Worker 的 `kv_cache_specs`
 - 输入每个 Worker 的 `available_memory`
 - 返回 `kv_cache_configs: list[KVCacheConfig]`
 - `kv_cache_configs[i]` 对应 global worker rank `i`
 - b. 如果 auto-fit 改变了 `max_model_len`, `collective_rpc("update_max_model_len", ...)` 将新值同步给所有 Worker。
 - c. `scheduler_kv_cache_config = generate_scheduler_kv_cache_config(...)` 调用点
 - `generate_scheduler_kv_cache_config()`

- 断言所有配置的 `num_blocks` 相同
- `deepcopy(kv_cache_configs[0])`
- 将 `UniformTypeKVCacheSpecs` 收敛为代表性 Spec
- 返回 `scheduler_kv_cache_config`

规划过程主要完成：

1. 合并不同 Worker 的 layer spec，得到全模型层集合；
2. 生成全局 KV Cache groups；
3. 把全局 groups 投影回每个 Worker，处理 PP 分片；
4. 根据每个 Worker 的显存生成各自配置；
5. 把所有 Worker 的 `num_blocks` 对齐到最小值，保证共享 Scheduler 可以使用一致的 block ID 空间。

这里不是把所有 Worker 配置合并成一个新对象。Worker 配置已经在前一步对齐；Scheduler 取第 0 份配置作为逻辑代表，并去掉 Scheduler 不需要保留的逐层 Uniform Spec 差异。

注意顺序：auto-fit 同步发生在生成 Scheduler 代表配置之前，不是在 Worker 分配 KV Cache 之后补做。

阶段 8：Worker 分配 KV Cache 并完成 Warmup

规划得到 `list[KVCacheConfig]` 后，`_initialize_kv_caches()` 继续调用 Executor，把对应配置分发给每个 Worker。

- `EngineCore._initialize_kv_caches()`
 - `self.model_executor.initialize_from_config(kv_cache_configs)` 调用点
 - `Executor.initialize_from_config()`
 1. `collective_rpc("initialize_from_config", args=(kv_cache_configs,))`
 - 跨 RPC 进入每个 Worker: `WorkerWrapperBase.initialize_from_config(kv_cache_configs)`
 - `config = kv_cache_configs[self.global_rank]`
 - `self.worker.initialize_from_config(config)`
 - `gpu_worker.Worker.initialize_from_config(single_config)`
 - `cache_config.num_gpu_blocks = config.num_blocks`

- `ensure_kv_transfer_initialized(...)`
- `GPUModelRunner.initialize_kv_cache(config)`
 - `initialize_kv_cache_tensors()` 分配 backing tensors、生成逐层 views。
 - `bind_kv_cache()` 把 layer name 对应的 view 绑定到模型层和 runner 状态。

2. 第一个 collective RPC 全部返回后，再调用 `collective_rpc("compile_or_warm_up_model")`

- 跨 RPC 进入每个 Worker: `compile_or_warm_up_model()`，完成 model compilation、kernel warmup 和 CUDA Graph preparation。

EngineCore 将完整的 `list[KVCacheConfig]` 作为 RPC 参数发给每个 Worker wrapper。 `WorkerWrapperBase.initialize_from_config()` 根据自己的 `global_rank` 选出单份配置，真实 Worker 最终只消费自己的 `KVCacheConfig`。

`Executor.initialize_from_config()` 返回时已经完成两个全局 barrier:

1. 所有 Worker 完成 KV 物理存储初始化；
2. 所有 Worker 完成 `compile_or_warm_up_model()`。

KV 物理存储的下一层实现分别位于 V1 `GPUModelRunner.initialize_kv_cache()` 和 V2 `GPUModelRunner.initialize_kv_cache()`。

阶段 9：构造 Scheduler 侧逻辑资源

所有 Worker 完成 KV Cache 分配和 Warmup 后，`_initialize_kv_caches()` 返回 Scheduler 代表配置； `EngineCore.__init__()` 接着构造 Scheduler 的 CPU 侧逻辑资源。

```

1  Executor.initialize_from_config() 返回
2  → EngineCore._initialize_kv_caches() 返回 scheduler_kv_cache_config
3  → EngineCore.__init__() 从下一行继续

```

- `EngineCore.__init__()` 从 KV 初始化之后继续
 - `StructuredOutputManager(vllm_config)`
 - `Scheduler = scheduler_config.get_scheduler_cls()`
 - `resolve_kv_cache_block_sizes(...)`
 - `self.scheduler = Scheduler(...)` 调用点

- `Scheduler.__init__()`
 - `KVCacheManager(...)`
 - `KVCacheCoordinator`
 - `BlockPool`
 - 建立 CPU 侧 block 元数据和所有权体系

这一步不会再次申请 GPU KV Cache 显存：

- Worker 已经建立 GPU KV tensors；
- Scheduler 在 CPU 侧建立 request、logical block、block table 和物理 `block_id` 所有权相关的元数据；
- 两侧通过相同的 KV Cache 规划和 block ID 空间，在运行期共同解释 Scheduler 输出。

如果配置了 KV Connector，`EngineCore.__init__()` 在 Scheduler 构造后还会收集 Worker connector 的 handshake metadata。这是可选路径，不改变“先完成 Worker KV 初始化，再构造 Scheduler，最后发送 Engine Ready”的基本顺序。

四、Engine Ready 与 HTTP Serving

阶段 10：完成两种 Engine Ready 同步

`EngineCore.__init__()` 完成 Scheduler 和可选 connector 状态后返回；外层 `EngineCoreProc` 随即启动 I/O threads，并向 Frontend 发布 Ready 信息。

- `EngineCoreProc.__init__()` 在 `super().__init__()` 返回后继续
 - a. `Thread(target=self.process_input_sockets, ...)`
 - 跨线程进入 `process_input_sockets()`
 - 创建 `EngineCoreReadyResponse`
 - 通过每个 input DEALER socket 发送 payload
 - API Server 的 input ROUTER 可以收到该 payload
 - `ready_event.set()`
 - b. `Thread(target=self.process_output_sockets, ...)`。
 - c. `ready_event.wait()` 返回后，`with _perform_handshakes(...)` 的主体结束。
 - 内层 `_perform_handshake()` context 退出并发送 handshake READY。
 - d. `EngineCoreProc.__init__()` 返回到 `run_engine_core()`。

EngineCore input thread 连接的是 Frontend 的 input ROUTER。它发送的第一条消息既用于让 ROUTER 认识 EngineCore identity，也携带启动后的最终配置：

```
1 EngineCoreReadyResponse
2   |─ max_model_len
3   |─ num_gpu_blocks
4   |─ block_size
5   |─ dp_stats_address
6   |─ kv_cache_size_tokens
7   |─ kv_cache_max_concurrency
8   |─ dtype
9   |─ world_size
10  |─ data_parallel_size
11  |─ vllm_version
```

API 进程先完成 EngineCoreClient 的构造返回链：

- `launch_core_engines()` 退出 context 后调用 `wait_for_engine_startup()`
 - `wait_for_engine_startup()` 接收 handshake READY。
 - `launch_core_engines()` 返回到 `MPCClient.__init__()`。
- `MPCClient.__init__()` 从 `with launch_core_engines(...)` 之后继续
 - 建立 encoder 和 decoder。
 - `input ROUTER 接收 EngineCoreReadyResponse`
 - `_apply_ready_response(payload)`
 - 同步 `max_model_len`
 - 累加或设置 `num_gpu_blocks`
 - 同步 `block_size`
 - 前端尚未设置相关值时，同步 KV Cache capacity

随后同步构造调用逐层返回：

```
1 MPCClient.__init__()
2   → AsyncMPCClient.__init__()
3   → EngineCoreClient.make_async_mp_client()
4   → AsyncLLM.__init__()
5   → AsyncLLM.from_vllm_config()
6   → build_async_engine_client_from_engine_args()
```

`build_async_engine_client_from_engine_args()` 完成 `reset_mm_cache()` 后 `yield`；外层 context manager 再把同一个 `engine_client` `yield` 给 `run_server_worker()`。此时才进入 HTTP 应用启动链：

- `build_async_engine_client_from_engine_args()` : `reset` 后 `yield`
 - 返回外层 `build_async_engine_client()` 的 `yield`
- `run_server_worker()` 的 `async with` 主体恢复执行
 - `shutdown_task = await build_and_serve(...)`
 - `build_and_serve()`
 - `get_supported_tasks()`
 - `build_app(...)`
 - `init_app_state(...)`
 - `serve_http(...)` → HTTP Server 开始服务。

发送顺序和消费顺序不是同一个概念：

- `process_input_sockets()` 先发送 `EngineCoreReadyResponse` ；
- `EngineCoreProc.__init__()` 内的 `handshake context` 随后退出并发送 `handshake READY`，之后构造函数才返回；
- API 先在 `launch_core_engines()` 中等待 `handshake READY`；
- 然后 `MPCClient` 再读取已经可能缓存在 `input ROUTER` 中的 `EngineCoreReadyResponse` 。

三种 READY 不要混淆

名称	通道	发送时机	准确含义
<code>WorkerProc</code> <code>READY</code>	Worker ready pipe	Worker 构造、设备初始化、模型加载和 MQ 创建完成	Queue handles 已交给 Executor；仍需完成 MQ readiness，同样尚未规划和分配 KV Cache

handshake {status: READY}	临时 handshake socket	EngineCoreProc.__init__() 内部完成主体初始化和 I/O thread ready, 退出 handshake context 时	wait_for_engine_startup() 可以返回; 随后 EngineCoreProc.__init__() 才返回
EngineCoreReadyResponse	EngineCore input DEALER → API input ROUTER	EngineCore 初始化完成, input thread 建立	将最终模型长度、block 数量、block size 和容量信息同步给 Frontend

可以把三者分别理解为：

```

1 Worker process constructed
2 → EngineCore process initialized
3 → Frontend receives final initialization results

```

它们处于不同层级，不能统一写成含义模糊的“Worker ready”或“Engine ready”。

五、进入请求运行期

阶段 11: EngineCore busy loop

EngineCore 侧，EngineCoreProc(...) 构造返回后，调用栈回到 EngineCoreProc.run_engine_core()，它安装 signal handler 后直接调用 engine_core.run_busy_loop()。API 侧则在完成 ReadyResponse 同步后进入 build_and_serve()。这两条调用链在不同进程中并行推进；等到 HTTP 请求到来时，EngineCore busy loop 已经在等待工作。

- HTTP endpoint handler（省略具体 OpenAI endpoint 的参数转换）
 - AsyncLLM.generate(...)
 - AsyncLLM.add_request(...)
 - AsyncLLM._add_request(...)
 - engine_core.add_request_async(request)
 - 跨进程、跨线程进入 EngineCoreProc.process_input_sockets()
 - input_queue.put_nowait((request_type, request))

- `EngineCoreProc.run_engine_core()`
 - `engine_core.run_busy_loop()` 调用点
 - `EngineCoreProc.run_busy_loop()`
 1. `_process_input_queue()` 将请求交给 EngineCore / Scheduler 状态。
 2. `_process_engine_step()`
 - `self.step_fn()` 由 `EngineCore.__init__()` 选择：
 - 普通路径 → `EngineCore.step()`
 - `Scheduler.schedule()`
 - `model_executor.execute_model(scheduler_output)` → Worker execution boundary
 - `Scheduler.update_from_output(...)`
 - 启用 batch queue 的路径 → `EngineCore.step_with_batch_queue()`。
 - `EngineCoreOutputs` 放入 output queue，再由 output thread 发回 API Server Process。

到这里，启动期建立的配置和资源已经成为运行时基础。普通 steady-state 请求路径中，持续变化的是：

- Request 的 waiting、running 和 finished 状态；
- logical block 到物理 `block_id` 的映射；
- 每个请求的 Block Table；
- 每轮 SchedulerOutput、ModelRunnerOutput 和 EngineCoreOutputs。

六、部署分支速查

本文没有展开下面这些分支，但查源码时需要先判断当前配置是否已经偏离主线。

变化	主要影响
<code>world_size=1</code> 、 <code>backend='uni'</code>	Worker 角色位于 EngineCore 进程内，没有 WorkerProc READY
<code>backend='mp'</code>	EngineCore 创建多个 WorkerProc，并增加 Worker/MQ readiness 同步

backend=`ray`	Worker 和 Executor 的创建、通信与生命周期交给 Ray 路径
<code>data_parallel_size>1</code>	可能出现多个 EngineCore、DP Coordinator, 以及 <code>DPAsyncMPCClient</code> / <code>DPLBAsyncMPCClient</code>
多 API Server	Frontend 的地址管理和 EngineCore 共享关系发生变化
headless	不创建正常 HTTP API Server, 由外部 Frontend 与 EngineCore 配合
gRPC	CLI 直接选择 <code>serve_grpc()</code> , 不进入本文的 HTTP 主线
V2 Model Runner	V1 Engine 不变, 但 Worker 内部 KV Cache 与执行准备实现切换到 <code>vllm.v1.worker.gpu.model_runner.GPUModelRunner</code>

七、按问题查源码

想查的问题	首先看哪里
<code>vllm serve</code> 最终选择了哪个入口?	<code>ServeSubcommand.cmd()</code>
HTTP socket 在什么时候建立?	<code>setup_server()</code> 与 <code>run_server()</code>
<code>VllmConfig</code> 在哪里产生?	<code>build_async_engine_client_from_engine_args()</code>
EngineCore 进程在哪里创建?	<code>launch_core_engines()</code>
HELLO/READY 协议在哪里?	<code>wait_for_engine_startup()</code> 、 <code>_perform_handshake()</code>
Worker 进程在哪里创建并加载模型?	<code>MultiprocExecutor._init_executor()</code> 、 <code>WorkerProc</code>
KV Cache 可用显存在哪里测量?	<code>Worker.determine_available_memory()</code>

每个 Worker 的 KVCacheConfig 在哪里生成?	<code>get_kv_cache_configs()</code>
Worker 在哪里分配 GPU KV Cache?	<code>Worker.initialize_from_config()</code>
Scheduler 在哪里建立 BlockPool?	<code>Scheduler.__init__()</code>
Frontend 在哪里拿到最终容量?	<code>process_input_sockets()</code> 、 <code>MPClien t._apply_ready_response()</code>
HTTP 服务何时真正开始?	<code>build_and_serve()</code>

八、定位任意初始化函数的方法

完整主线可以压缩为：

▼

LaTeX |

```

1  CLI dispatch
2  → build VllmConfig and bind socket
3  → construct AsyncLLM and MPCClient
4  → launch and handshake with EngineCore
5  → construct Executor and Worker topology
6  → initialize devices and load model weights
7  → collect KVCacheSpec and profile available memory
8  → build per-Worker KVCacheConfig
9  → allocate Worker KV tensors and warm up execution
10 → construct Scheduler logical block state
11 → synchronize final Ready metadata
12 → start HTTP serving
13 → enter request-time EngineCore loop

```

遇到一个不熟悉的初始化函数时，按下面五个问题定位：

1. 它运行在 API Server、EngineCore，还是 Worker 边界？
2. 它的直接调用者是谁？
3. 它返回之前要等待哪个进程、Worker 或 barrier？
4. 它产生的是配置、GPU 物理存储，还是 CPU 运行时逻辑状态？
5. 它的结果下一步由谁消费？

只要这五个问题能够回答，就可以把函数放回整条启动链，而不必依赖它所在的文件名猜测职责。